



Capacity Planning for Black Friday Sale — Deep, Practical, and End-to-End

This guide is engineered for **hands-on execution**: forecasting → modelling → sizing → configs → test plans → dashboards → chaos → war-room. It includes **formulas, step-by-step calculations, YAML/INI snippets, SQL, and case studies** you can adapt immediately.

0) SLA, SLO, and Performance Budget (Set These First)

Example target at peak (15:00–22:00 IST):

- **Availability (API) $\geq 99.95\%$**
- **Checkout P95 ≤ 800 ms, P99 ≤ 1.6 s**
- **Catalog P95 ≤ 400 ms**
- **Payment success rate $\geq 98.5\%$**
- **Error rate (5xx) $< 0.5\%$**
- **Cart loss $< 0.2\%$**

These numbers drive **capacity** and **test pass/fail**.

1) Demand Forecasting (with Working Math)

Inputs (from last 3–5 years + campaigns):

- 2024 peak hour: **720k requests** (HTTP) in 60 min
- Expected YoY uplift: **+35%**
- Extra uplift from new marketing: **+20%**
- Traffic blend: **Read 82% / Write 18%**
- Mobile share: **65%**, Web: **35%**

Projected 2025 peak hour requests

Base = 720,000

YoY = $720,000 \times 1.35 = 972,000$

Campaign = $972,000 \times 1.20 = 1,166,400$ requests/hour

Peak TPS

TPS_{peak} = $1,166,400 / 3,600 = 323.999... \approx 324$ TPS (aggregate across endpoints)

Real sites see **multi-modal peaks** (browse spikes; checkout bursts). Plan a **buffer $\geq 50\%$** .

TPS_{planned} = $324 \times 1.5 = 486$ TPS

Per-tier mix example

- Catalog/Search **70%** $\rightarrow 340$ TPS
 - Cart/Checkout **20%** $\rightarrow 97$ TPS
 - Auth/Profile/Other **10%** $\rightarrow 49$ TPS
-

2) Concurrency & Little's Law

Little's Law: $WIP = \lambda \times W$

- For **Checkout**: Target P95 = **800 ms**; assume Avg = **400 ms**
Concurrency_checkout = $97 \text{ TPS} \times 0.4 \text{ s} = 38.8 \approx 39$ in-flight
- For **Catalog**: Avg = **150 ms**
Concurrency_catalog = $340 \times 0.15 = 51$
- For **All** (avg latency 200 ms assumption)
Concurrency_total = $486 \times 0.2 = 97.2 \approx 100$

Use these to right-size **thread pools**, **DB pools**, and **connection limits**.

3) CPU Core Sizing by Workload (Deterministic Method)

Rule of thumb:

Cores_needed = $(\text{CPU_ms_per_request} \times \text{TPS}) / (1000 \times \text{Target_Util})$

Example (per request CPU on app tier):

- Catalog: **12 ms** CPU/req $\rightarrow 12 \times 340 = 4,080 \text{ ms/s} \rightarrow$ **4.08 cores @100%**
At **65% target util** $\rightarrow 4.08 / 0.65 = 6.28 \rightarrow 7$ cores
- Checkout: **30 ms** CPU/req $\rightarrow 30 \times 97 = 2,910 \rightarrow$ **2.91 cores** $\rightarrow /0.65 = 4.48 \rightarrow 5$ cores

- Other: $15 \text{ ms} \times 49 = 735 \text{ ms/s} \rightarrow 0.735 \text{ cores} \rightarrow /0.65 \approx 1.13 \rightarrow 2 \text{ cores}$

App tier total $\approx 7 + 5 + 2 = 14$ cores (round up, add 50% growth buffer $\rightarrow 21$ cores).

If a pod has **1 vCPU**, plan **21 pods**; if **500m CPU/pod**, plan **42 pods** (with HPA headroom).

4) Memory Sizing (Sessions, Caches, Queues)

4.1 Session/Heap

- Average live objects/session: **200 KB**
- Peak concurrent sessions (logged-in): **30,000**

$\text{Memory_sessions} = 200 \text{ KB} \times 30,000 = 6,000,000 \text{ KB} \approx 5.72 \text{ GB}$

Add **overheads (class metadata, GC, TLS)** $\sim 40\% \rightarrow 8 \text{ GB}$ across app tier. Spread across pods; validate with **heap histograms**.

4.2 Redis Cart Cache

- Cart JSON avg **6.5 KB** (post-compression $\sim 4.0 \text{ KB}$)
- Active carts concurrently **400k**

$\text{Raw} = 4 \text{ KB} \times 400,000 = 1,600,000 \text{ KB} \approx 1.53 \text{ GB}$

Overhead (keys, pointers, fragmentation) $\approx \times 2 \rightarrow \sim 3.1 \text{ GB}$

Cluster with 3 shards $\rightarrow \sim 1.1 \text{ GB/shard}$ (min 4 GB RAM per shard for headroom)

5) Bandwidth & LB Sizing

Assume average payload sizes:

- Catalog GET **response $\sim 35 \text{ KB}$**
- Checkout POST **response $\sim 8 \text{ KB}$, request $\sim 2 \text{ KB}$**
- Static assets served via **CDN** (not counted here)

Catalog egress/sec: $340 \text{ TPS} \times 35 \text{ KB} = 11,900 \text{ KB/s} \approx 11.6 \text{ MB/s}$
 $\approx 92.8 \text{ Mb/s}$ ($\times 1.2$ headroom) $\rightarrow 112 \text{ Mb/s}$

Checkout bi-directional:

Requests: $97 \times 2 \text{ KB} = 194 \text{ KB/s}$

Responses: $97 \times 8 \text{ KB} = 776 \text{ KB/s}$

Total: $\sim 970 \text{ KB/s} \approx 0.95 \text{ MB/s} \approx 7.6 \text{ Mb/s}$ ($\times 1.5$ headroom) $\rightarrow \sim 11.4 \text{ Mb/s}$

Sum + other endpoints $\rightarrow$ size NICs/LBs $\geq 2\times$ the computed peak.

6) Database & Pooling Math (PostgreSQL/Aurora)

6.1 Queries per Transaction

- Checkout flow $\approx 6 \text{ SQL ops/txn}$ (3 reads, 3 writes)
- Catalog $\approx 2 \text{ reads/req}$

Peak QPS

- Catalog: $340 \text{ TPS} \times 2 = 680 \text{ QPS}$ (reads)
- Checkout reads: $97 \times 3 = 291 \text{ QPS}$ (reads)
- Checkout writes: $97 \times 3 = 291 \text{ QPS}$ (writes)

Split

- Reads total $\approx 971 \text{ QPS} \rightarrow$ serves from **reader endpoints**
- Writes total $\approx 291 \text{ QPS} \rightarrow$ to **writer**

6.2 Connection Pool Size

Pool $\approx$ Concurrency $\times$ (DB_time / End-to-end_latency) $\times$ Safety

For Checkout:

- Concurrency ~ 39
- DB_time $\sim 120 \text{ ms}$, E2E $\sim 400 \text{ ms}$
- Safety factor **1.3**

Pool_checkout $\approx 39 \times (0.12 / 0.4) \times 1.3 = 39 \times 0.3 \times 1.3 \approx 15.21 \rightarrow 16$

Round by service instance; cap total to prevent saturation.

6.3 Key PostgreSQL Settings (writer)

postgresql.conf (illustrative)

max_connections = 800

shared_buffers = 8GB

effective_cache_size = 24GB

work_mem = 32MB

maintenance_work_mem = 1GB

wal_compression = on

checkpoint_completion_target = 0.9

random_page_cost = 1.1

effective_io_concurrency = 200

autovacuum_vacuum_cost_limit = 4000

Use PgBouncer (transaction pooling) to keep backend connections low:

pgbouncer.ini

[databases]

shop = host=aurora-writer port=5432 dbname=shop user=app

[pgbouncer]

pool_mode = transaction

max_client_conn = 5000

default_pool_size = 100

reserve_pool_size = 50

7) Kafka / MQ Sizing

Assume **order-created** topic:

- Events/txn: **1**
- TPS_orders = **97**
- Message size avg: **1.2 KB**

Ingress throughput: $97 \times 1.2 \text{ KB} \approx 116.4 \text{ KB/s}$ ($\sim 0.11 \text{ MB/s}$)

Small for bandwidth; the constraint is **latency & lag**.

Partitions: For sub-100 ms end-to-end,

- Target $\leq 10 \text{ ms}$ processing per message per consumer instance
- If one consumer instance can do **500 msg/s**, need $97 / 500 \approx 0.194 \rightarrow 1 \text{ partition}$ is enough; for headroom and parallelization, choose **6 partitions** ($2\times\text{--}4\times$ expected consumer instances is healthy).

Broker snippet:

```
num.partitions=6
```

```
message.max.bytes=1048576
```

```
log.retention.hours=72
```

```
unclean.leader.election.enable=false
```

```
min.insync.replicas=2
```

Consumer lag alert: **>1000 messages** or **>500 ms** for **>2 min**.

8) Edge, CDN, Rate Limiting & Backoff

NGINX (rate limit + burst + circuit-ish)

```
http {
```

```
    limit_req_zone $binary_remote_addr zone=rl:10m rate=100r/s;
```

```
server {
```

```
    location /api/ {
```

```
        limit_req zone=rl burst=200 nodelay;
```

```
        proxy_connect_timeout 1s;
```

```
        proxy_read_timeout 2s;
```

```
        proxy_send_timeout 2s;
```

```
    }
```

```
}  
}
```

Resilience4j (Spring Boot) – backoff with jitter

resilience4j.retry.instances.payments.max-attempts=4

resilience4j.retry.instances.payments.wait-duration=200ms

resilience4j.retry.instances.payments.enable-exponential-backoff=true

resilience4j.retry.instances.payments.exponential-backoff-multiplier=2

resilience4j.retry.instances.payments.retry-exceptions=com.foo.TimeoutException

resilience4j.timelimiter.instances.payments.timeout-duration=1200ms

Jitter formula (full jitter):

*sleep = random(0, base * 2^attempt)*

Example: base=200 ms → attempts 1..3 sleeps in [0..400], [0..800], [0..1600] ms.

9) Kubernetes Autoscaling & Quotas

HPA (CPU + custom latency via Prometheus adapter)

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: checkout-api

spec:

minReplicas: 8

maxReplicas: 40

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 65

- type: Pods

pods:

metric:

name: http_p95_latency_ms

target:

type: AverageValue

averageValue: "700" # scale out if P95 > 700ms

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: checkout-api

Pod Requests/Limits (avoid CPU throttling spikes)

resources:

requests:

cpu: "500m"

memory: "1Gi"

limits:

cpu: "1"

memory: "2Gi"

10) Caching Strategy (Edge + App)

- **CDN:** Pre-warm top **N** URLs (home, category, search facets, bestsellers).
Soft-TTL/Serve-stale on backend errors.

- **App:** Two-tier cache: **in-proc (Caffeine)** for sub-millisecond hot keys; **Redis** for cross-pod.

Caffeine example (Spring):

@Bean

```
public Cache<String, Product> productCache() {
    return Caffeine.newBuilder()
        .maximumSize(500_000)
        .expireAfterWrite(Duration.ofMinutes(15))
        .recordStats()
        .build();
}
```

Redis eviction: volatile-lru for TTL-ed keys; keep hit-rate >95%.

11) Performance Test Plan (Black Friday Drill)

11.1 Mix & Phases

- **Warmup:** 15 min to load caches
- **Load:** 2 h at **100% TPS_planned (486)**
- **Spike:** 10 min at **150% (729 TPS)** within 30 s
- **Soak:** 8 h at **70% (340 TPS)** for leaks
- **Failover:** Kill one AZ + payment gateway partial outage

11.2 LG Sizing (JMeter)

- One LG can safely drive **2,500–3,000 rps** for simple APIs; we'll be conservative: **1,500 rps**.
- Required for spike (729 rps): **1 LG** is enough; for **headroom + redundancy** → **3 LGs**.

Users calc (per script):

Users_needed = (TPS_target × Avg_ThinkTime+RespTime) / IterationsPerUser

Example for Catalog (Avg think+resp 1.2 s):

Users = $340 \times 1.2 = 408$ VUs (if 1 iter/sec/user)

11.3 JMeter CLI

```
jmeter -n -t bf_plan.jmx \  
-Jthreads.catalog=420 -Jrampup.catalog=600 \  
-Jthreads.checkout=120 -Jrampup.checkout=600 \  
-Jhost.api=api.shop.example \  
-l results/bf_$(date +%F_%H%M).jtl
```

Backend Listener (InfluxDB) (if allowed):

```
# user.properties
```

```
backend_influxdb.url=http://influxdb:8086/write?db=jmeter
```

```
backend_influxdb.metrics.sender=org.apache.jmeter.visualizers.backend.influxdb.HttpMetricsSender
```

12) Observability: Dashboards & Alerts

PromQL examples

API P95 latency (ms)

```
histogram_quantile(0.95, sum by (le,route)(  
  rate(http_server_request_duration_seconds_bucket{job="checkout"}[5m])  
)) * 1000
```

Error rate

```
sum(rate(http_server_requests_seconds_count{status=~"5..",job="checkout"}[5m]))  
/  
sum(rate(http_server_requests_seconds_count{job="checkout"}[5m])) * 100
```

Kafka lag

```
max(kafka_consumergroup_lag{consumergroup="order-consumer"})
```

Alert rules (YAML)

groups:

- name: blackfriday

rules:

- alert: CheckoutP95TooHigh

expr: histogram_quantile(0.95, sum by
(le,route)(rate(http_server_request_duration_seconds_bucket{job="checkout"}{3m}))) * 1000 >
800

for: 5m

labels: {severity: page}

annotations:

summary: "Checkout latency P95 > 800ms"

- alert: PaymentErrorBurst

expr: (sum(rate(http_server_requests_seconds_count{job="payment",status=~"5.."}{2m}))) /
sum(rate(http_server_requests_seconds_count{job="payment"}{2m}))) * 100 > 1.0

for: 2m

labels: {severity: page}

annotations:

summary: "Payment 5xx > 1% for 2m"

13) Failure Injection & Chaos (Must-Pass Scenarios)

1. Payment gateway 503 (30% of calls for 5 min):

- Expect **backoff + jitter**; no retry storms; switch to **backup PG** when CB opens.
- **SLO**: success $\geq 98.5\%$, Checkout P95 < 1.2 s.

2. DB Reader AZ loss: read latency +40% for 3 min.

- Expect read-routing to healthy replicas.

3. Redis shard failover: hit-rate dips; ensure **serve-stale** and quick regen.

4. Inventory API +300 ms latency:

- Ensure **timeout budgets** respected; UI shows “checking stock...” non-blocking.

14) Case Studies (Expanded)

14.1 Cart Stall Due to DB Locks (Write Hotspot)

- **Symptom:** Checkout P95 climbed from **700 ms** → **2.3 s** at 19:10.
- **Findings:** UPDATE cart set items=? where cart_id=? hot row; **lock wait ~450 ms** average.
- **Fix:** Switched to **append-only cart events + snapshot writer**, reduced lock time to **<20 ms**.
- **Capacity impact:** Write QPS unchanged (≈ 290), but worst-case latency dropped **>70%**.

SQL index example

```
CREATE INDEX CONCURRENTLY idx_cart_cartid ON cart(cart_id);
```

```
CREATE INDEX CONCURRENTLY idx_cart_updated_at ON cart(updated_at);
```

14.2 Retry Storm on Payment

- **Symptom:** 5xx burst; gateway rate-limited.
- **Bad logic:** fixed 250 ms retries $\times$ 5, synchronized across pods → **thundering herd**.
- **Fix:** **Exponential backoff + full jitter** (200, 400–800, 0–1600, 0–3200 ms), **circuit breaker**, **alternate PG** routing.
- **Result:** Gateway 5xx contained to **0.4%**; success rate **99.1%**.

14.3 CDN Cold Cache on Hero Images

- **Symptom:** Home page P95 **2.6 s** at 15:02 as traffic surged.
- **Fix:** Pre-warm top **500** assets; **serve-stale-on-error**; short **stale-while-revalidate**.
- **Result:** Home P95 stable **<700 ms** through peak.

CDN headers example

Cache-Control: max-age=600, stale-while-revalidate=120, stale-if-error=600

15) Security & Third-Party Protection

- **WAF** rules for common bots/scanners
 - **Rate limit per token/IP** (see NGINX above)
 - **Token bucket** on critical endpoints (checkout submit)
 - **Graceful degrade** recommendations/wishlist if 3P fails
-

16) Runbooks & War-Room Protocol

War-room roster: Incident Commander, App Lead, DB Lead, Network/CDN, SRE/Observability, Payments SME, Business owner.

T-0 Incident Template (first 5 minutes):

- Scope: endpoints affected, regions, start time
- SLOs breached
- Immediate actions (scale, CB open/close, route to backup)
- Status cadence: every **5 min**
- Customer comms owner assigned

Rollback/Feature-flag playbook: keep hot toggles for expensive queries, recommendation calls, heavy banners.

17) Final Readiness Checklist (Executable)

- **Capacity sheet** filled (TPS, cores, memory, bandwidth, pools)
- **HPA** tested in a pre-prod scale-test
- **Redis** shards + failover rehearsal
- **DB vacuum/ANALYZE** completed; slow query thresholds tuned
- **PgBouncer** enabled; pool limits per service verified
- **CDN pre-warm** job executed
- **Payment failover** drill passed (CB, retries, backup PG)
- **Chaos** scenarios 1–4 passed
- **Dashboards + alerts** green; paging on-call verified

- **Comms templates** in status page tool
 - **Change freeze** in effect
-

18) Quick Formula Cheat-Sheet

- **Peak TPS** = Requests/hour ÷ 3600 × peak_factor
 - **Concurrency** = TPS × Avg latency (s)
 - **Cores** = (CPU_ms_per_req × TPS) ÷ (1000 × TargetUtil)
 - **DB Pool (per service)** ≈ Concurrency × (DB_time/E2E_time) × 1.3
 - **Redis RAM** ≈ avg_bytes_per_key × keys × 2 (overhead)
 - **Bandwidth** (Mb/s) ≈ (TPS × bytes_per_resp × 8) ÷ 1,000,000
-

19) Example Files/Configs (copy-ready)

19.1 Spring Boot Timeouts

server.tomcat.accept-count=2000

server.tomcat.max-threads=400

spring.mvc.async.request-timeout=1500

http.client.connect-timeout=400

http.client.read-timeout=800

19.2 Resilience (Checkout calls)

resilience4j.circuitbreaker.instances.pg.sliding-window-size=50

resilience4j.circuitbreaker.instances.pg.minimum-number-of-calls=20

resilience4j.circuitbreaker.instances.pg.failure-rate-threshold=30

resilience4j.circuitbreaker.instances.pg.wait-duration-in-open-state=5s

19.3 Redis (redis.conf)

maxmemory 4gb

maxmemory-policy volatile-lru

20) Post-Event Analysis (Feed Next Year's Plan)

Collect and compare:

- **P50/P95/P99** per endpoint vs SLOs
- **Autoscaling events** vs latency; look for oscillations
- **Top 10 slow SQL**; identify missing indexes
- **Cache hit ratio** and **evictions**
- **Consumer lag** and **retries** (reason codes)
- **Downstream 3P** latency/error budgets

Update models (TPS, cores, pools) with **actuals + growth**.

One-Page guide (What to Actually Do This Week)

1. Fill the **capacity worksheet** with the formulas above → derive pods/cores/memory/pools.
 2. Apply **HPA + latency metric**, validate in scale test.
 3. Enable **PgBouncer**, set conservative **pool sizes** per service.
 4. **Pre-warm** CDN and **Redis**.
 5. Implement **backoff+jitter** + **circuit breakers** + **payment failover**.
 6. Run **spike + failover + chaos** drills; fix any fragile parts.
 7. Lock **dashboards + alerts**; rehearse **war-room**.
-